

API TESTING REPORT

Authorization & Role-Based Access Control (RBAC) — Day 3

Postman-Based Testing of the TaskTrackerApi Authorization Layer

1. Introduction

This report documents the functional testing of the authorization layer added to the TaskTrackerApi on Day 3, building on the JWT authentication implemented on Day 2. Testing focused on verifying that protected endpoints correctly reject unauthenticated and unauthorized requests, that role-based restrictions behave as expected, and that the foundation for policy-based authorization returns the correct status codes.

All requests were executed manually in Postman using an environment that captures the login JWT and reuses it as a Bearer token on subsequent requests, allowing the same collection to be run under three identities: no token, a User-role token, and an Admin-role token.

2. Objective

- Confirm that endpoints protected with [Authorize] reject requests without a valid token (401 Unauthorized).
- Confirm that an authenticated user lacking the required role is rejected with 403 Forbidden rather than 401.
- Confirm that a correctly authorized Admin token is accepted and the operation succeeds.
- Verify the same behavior extends to the new policy-based authorization endpoint.
- Document evidence of each test case, including expected vs. actual status codes.

3. Testing Environment

Item	Detail
Project	TaskTrackerApi
Technology	ASP.NET Core Identity, JWT Bearer Authentication, Role-Based & Policy-Based Authorization
Testing Tool	Postman (manual requests + environment variables for token capture)
Auth Roles Used	User, Admin
Date	August 11, 2026

4. Test Cases — Authorization & RBAC

The table below lists all test cases executed against the protected routes, the Admin-only user-deletion endpoint, and the new policy-based endpoint. Each case was run once under the identity indicated in the Authentication column.

TC	Test Case	Endpoint	Authentication	Expected	Result
TC01	Protected route without token	GET /api/Tasks	No Token	401	401
TC02	Protected route with invalid token	GET /api/Tasks	Invalid JWT	401	401
TC03	Protected route with User token	GET /api/Tasks	User JWT	200	200
TC04	Protected route with Admin token	GET /api/Tasks	Admin JWT	200	200

TC	Test Case	Endpoint	Authentication	Expected	Result
TC05	Admin endpoint without token	DELETE /api/Users/{id}	No Token	401	401
TC06	Admin endpoint with User token	DELETE /api/Users/{id}	User JWT	403	403
TC07	Admin endpoint with Admin token	DELETE /api/Users/{id}	Admin JWT	204	204
TC08	Policy endpoint without token	Policy endpoint	No Token	401	401
TC09	Policy endpoint with User token	Policy endpoint	User JWT	403	403
TC10	Policy endpoint with authorized Admin	Policy endpoint	Admin JWT + Permission	Success (200/204)	204

5. Test Summary

Metric	Value
Total Test Cases	10
Authorization Tests (401/403)	6
Success Scenarios (200/204)	4
Passed	10
Failed	0

6. Conclusion

All 10 test cases executed against the authorization layer returned the expected HTTP status codes. Requests without a token or with an invalid token were consistently rejected with 401 Unauthorized before reaching controller logic. Authenticated requests lacking the Admin role were correctly rejected with 403 Forbidden, confirming that authentication and authorization are being evaluated as separate, sequential checks. Requests carrying a valid Admin token completed successfully.

The policy-based endpoint (TC08–TC10) showed the same pattern as the role-based Admin endpoint, confirming that the named "CanManageTasks" policy is enforced correctly once the Permission claim is present on the token. This validates the authorization foundation built today and confirms the API is ready for additional, more granular policies in future iterations.

7. Screenshot Evidence

TC01 — Protected route without token — GET /api/Tasks (No Token)

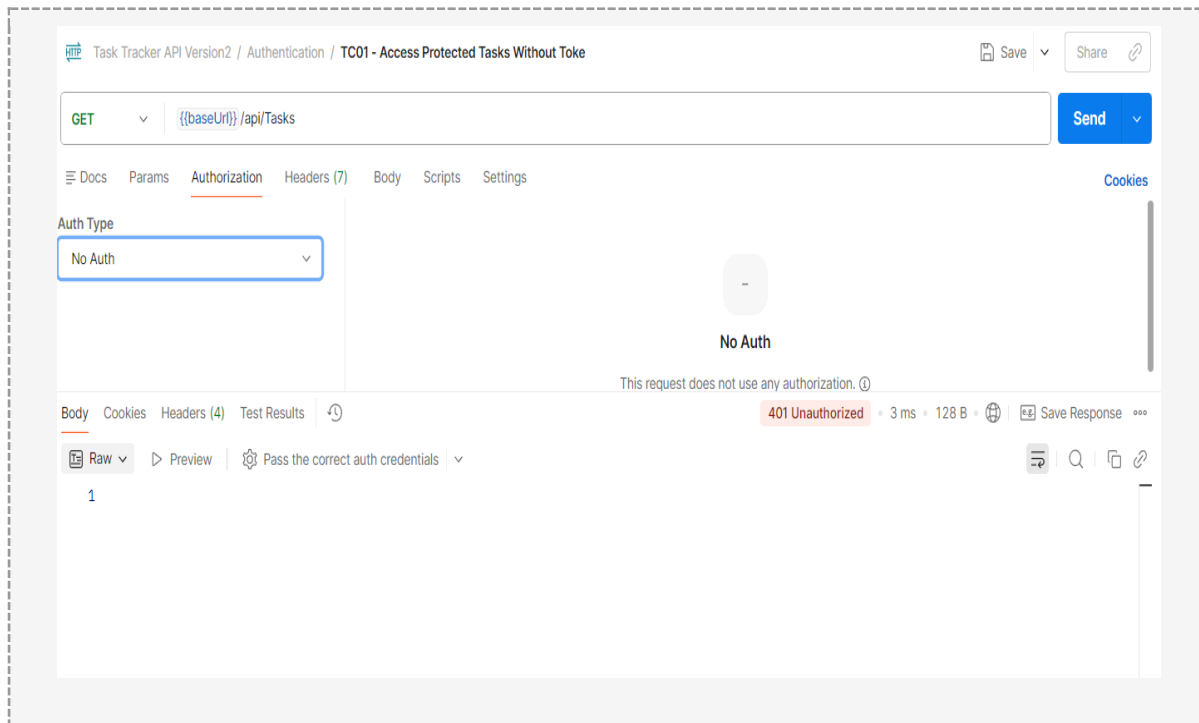


Figure: TC01 — Postman request/response

TC02 — Protected route with invalid token — GET /api/Tasks (Invalid JWT)

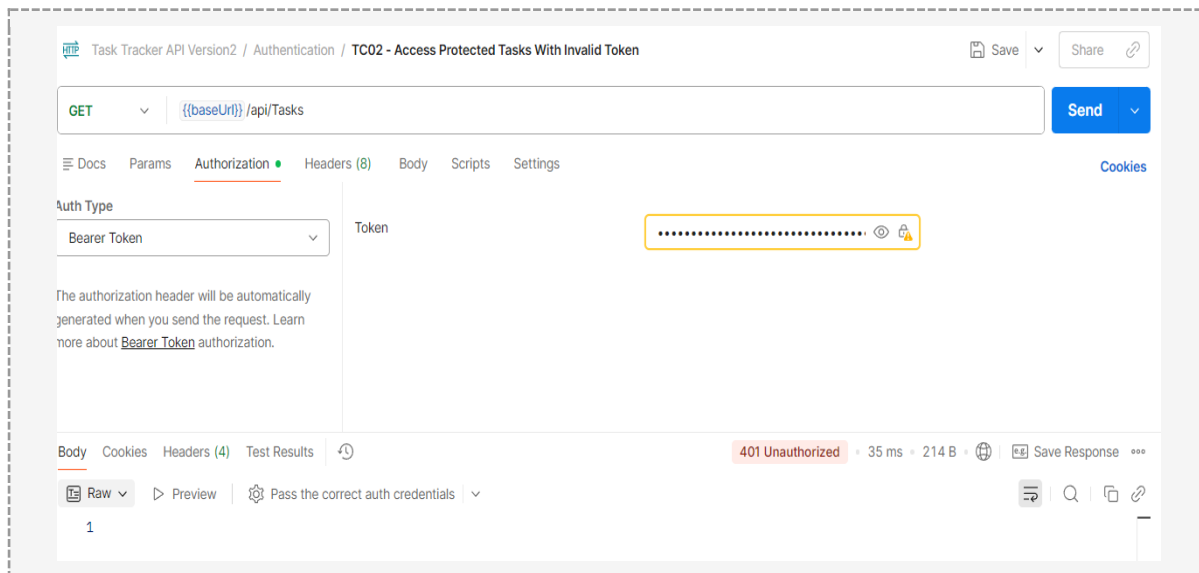


Figure: TC02 — Postman request/response

TC03 — Protected route with User token — GET /api/Tasks (User JWT)

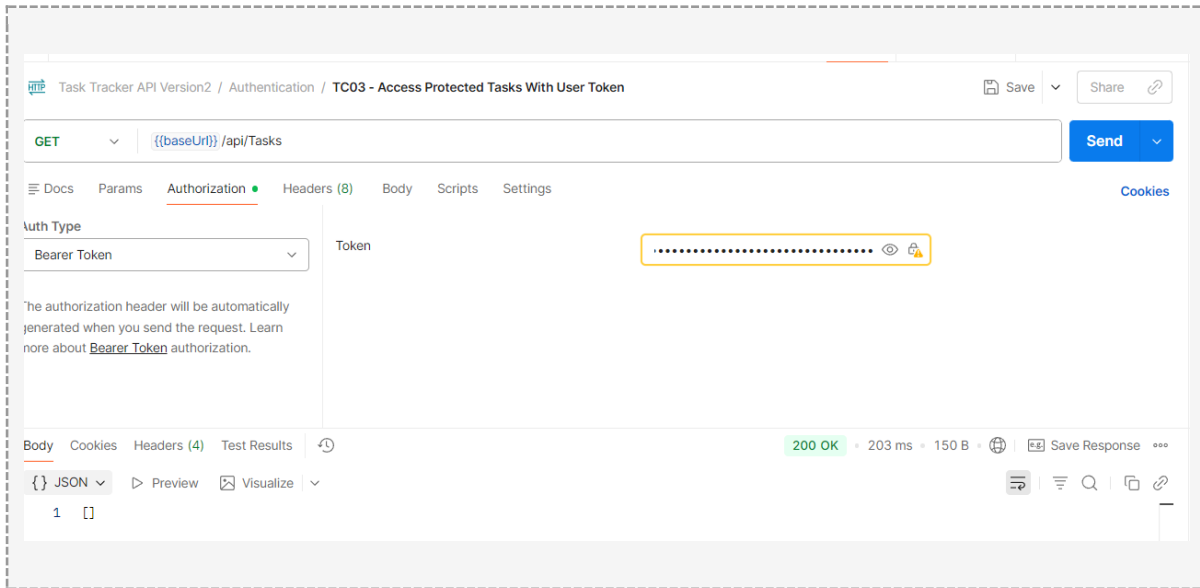


Figure: TC03 — Postman request/response

TC04 — Protected route with Admin token — GET /api/Tasks (Admin JWT)

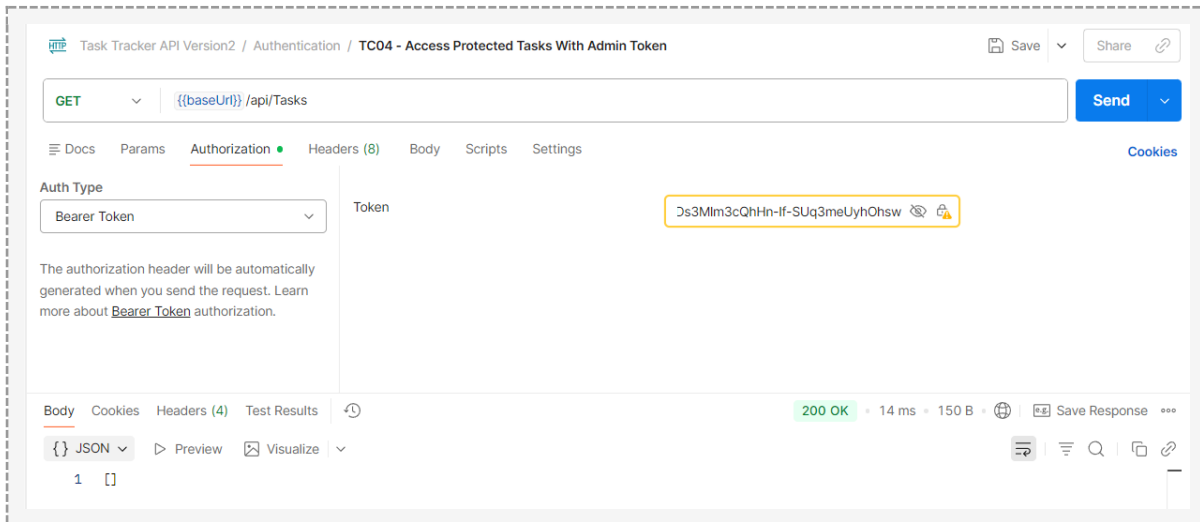
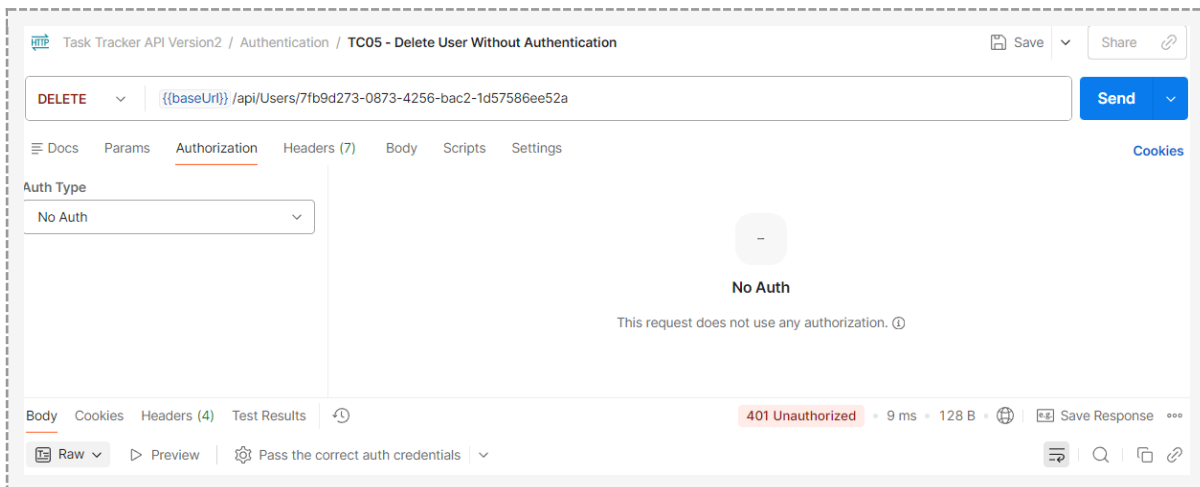


Figure: TC04 — Postman request/response

TC05 — Admin endpoint without token — DELETE /api/Users/{id} (No Token)



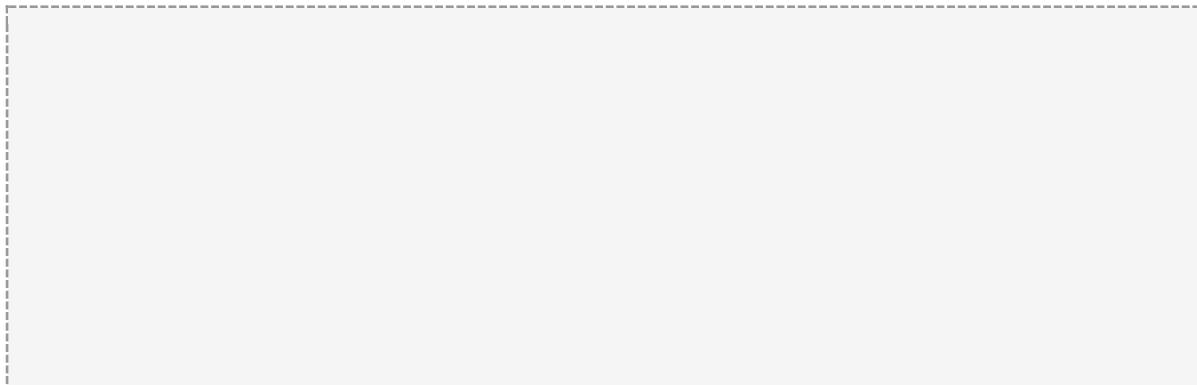


Figure: TC05 — Postman request/response

TC06 — Admin endpoint with User token — DELETE /api/Users/{id} (User JWT)

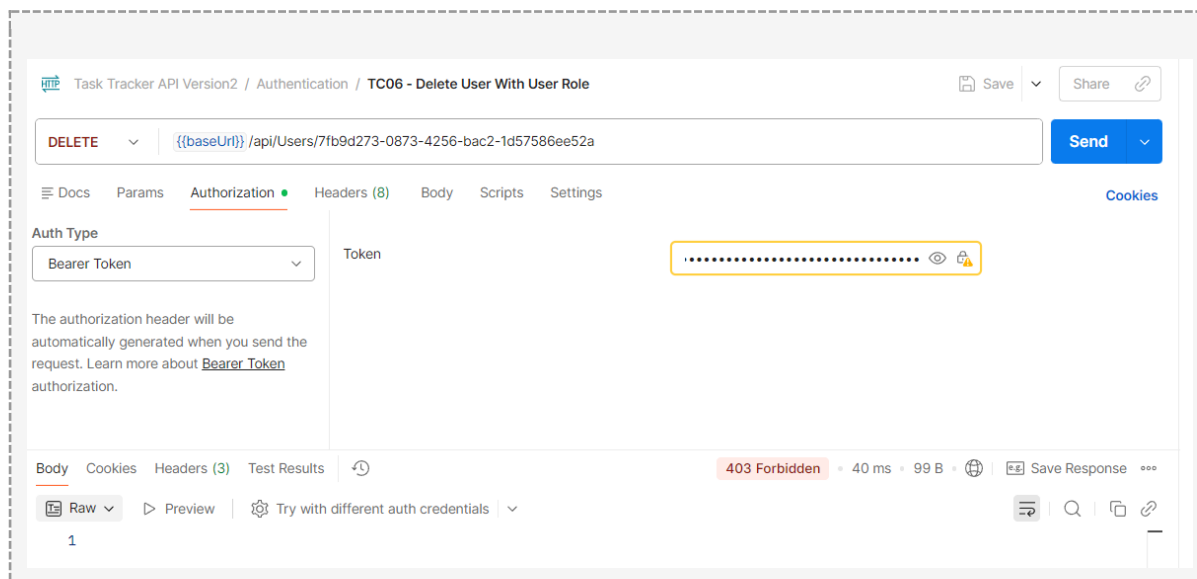


Figure: TC06 — Postman request/response

TC07 — Admin endpoint with Admin token — DELETE /api/Users/{id} (Admin JWT)

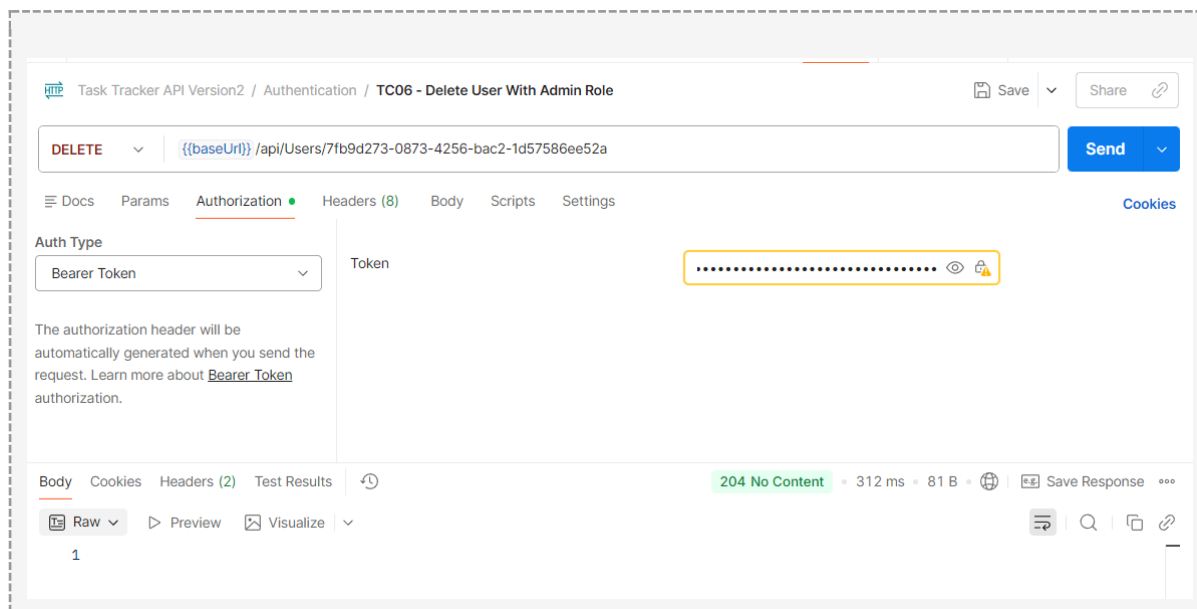


Figure: TC07 — Postman request/response

TC08 — Policy endpoint without token — No Token — Delete Task Without Authentication

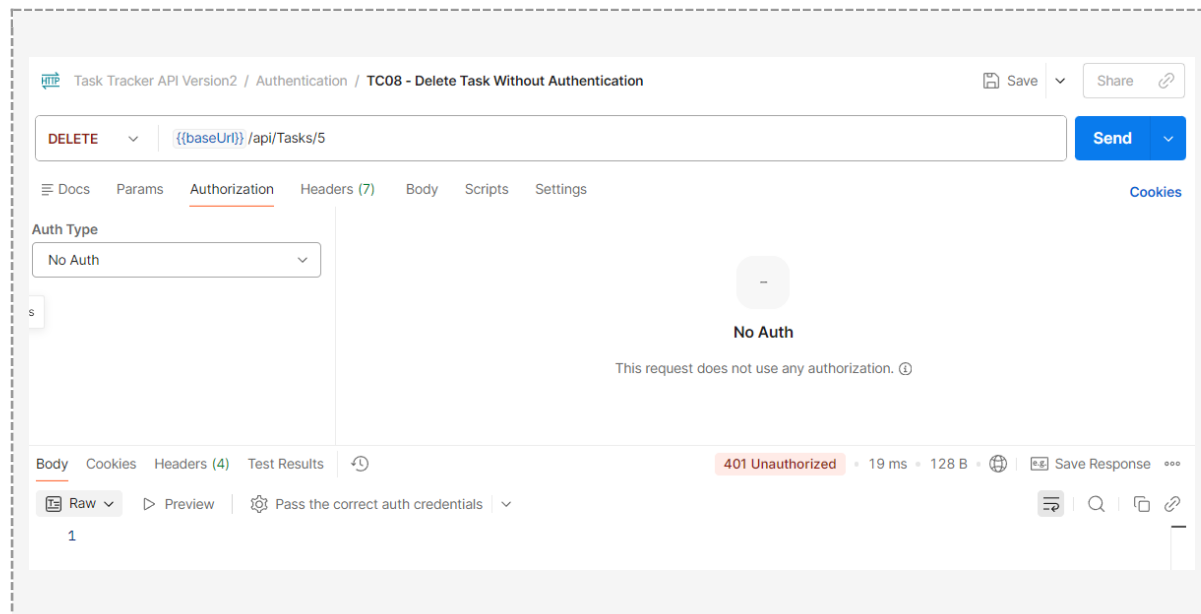


Figure: TC08 — Postman request/response

TC09 — Policy endpoint with User token — User JWT

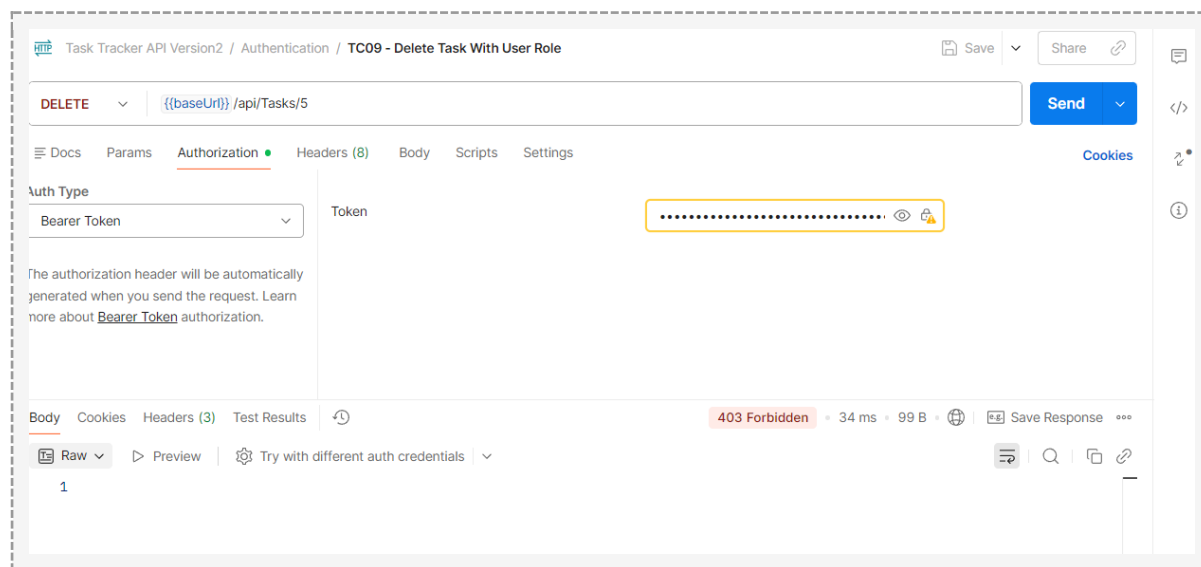
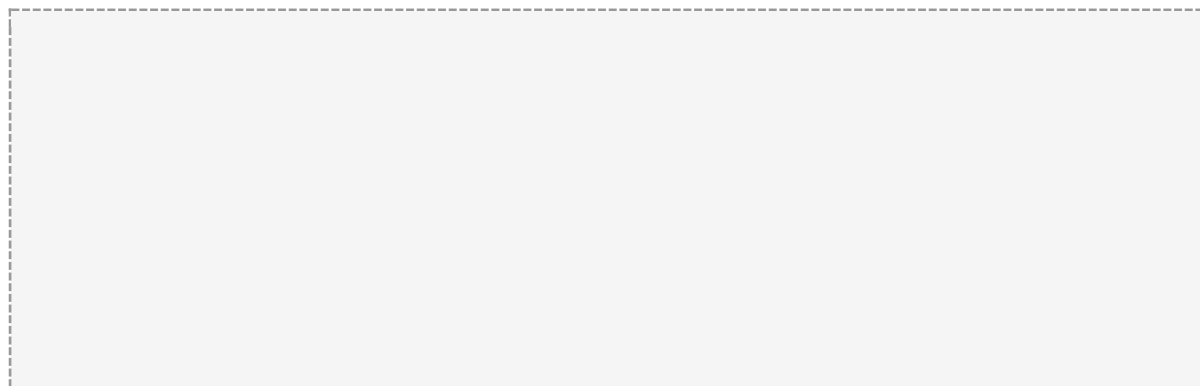


Figure: TC09 — Postman request/response

TC10 — Policy endpoint with authorized Admin — Admin JWT + Permission



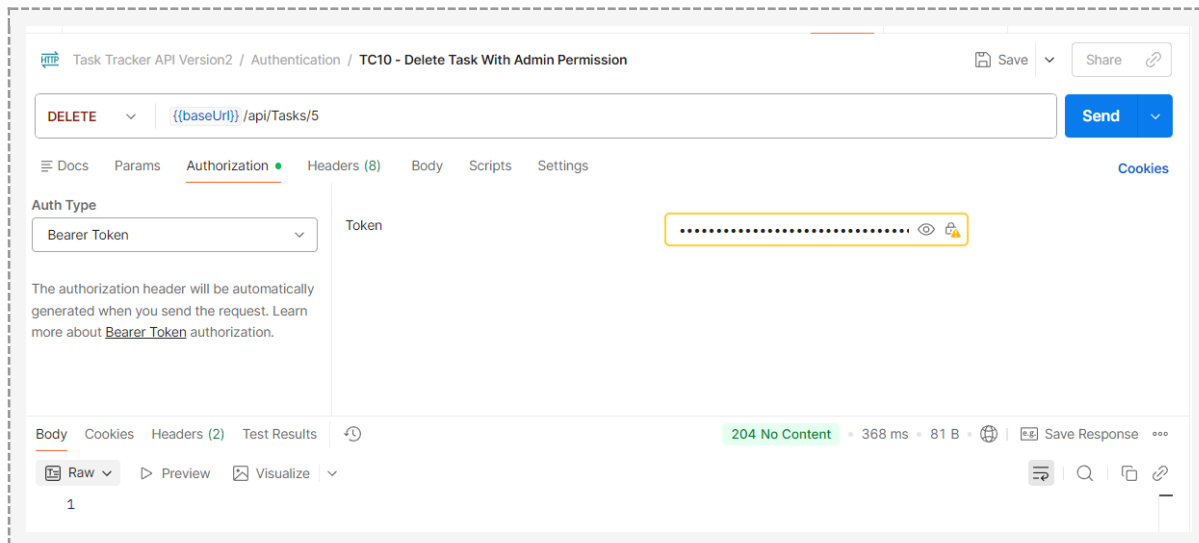


Figure: TC10 — Postman request/response